

High-Level Design

AI Patient Simulator — High-Level Design

1. Product Vision

The AI Patient Simulator is a multilingual, LMS-integrated clinical-skills simulation platform for psychotherapy, counselling, LI-CBT, ACT, CBT, and related training programmes.

The platform enables students to conduct simulated therapeutic conversations with AI patients/clients, receive structured competency-based feedback, review transcript-based evidence, and develop clinical skills over time.

For teachers, programme managers, and college managers, the system provides hierarchical dashboards, graph-based analytics, rubric reports, longitudinal student development tracking, operational support visibility, and controlled AI usage/cost monitoring.

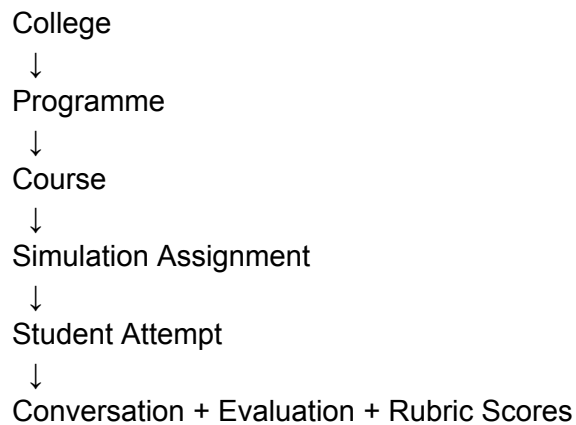
The system is not simply a chatbot. It is an educational simulation, assessment, supervision, analytics, support, and quality-assurance platform.

2. Core Platform Definition

A multilingual, LMS-integrated AI clinical-skills simulation platform for psychotherapy and LI-CBT/ACT training, with hierarchical access control, guided case authoring, ground-truth patient boundaries, dynamic emotional-state simulation, competency-linked rubrics, graph-based dashboards, transcript-based evaluation, student debriefing, technical support escalation, academic-safety protection, usage governance, and longitudinal development tracking across student, course, programme, and college levels.

3. Core Educational and Organisational Hierarchy

The platform must recognise the following hierarchy:

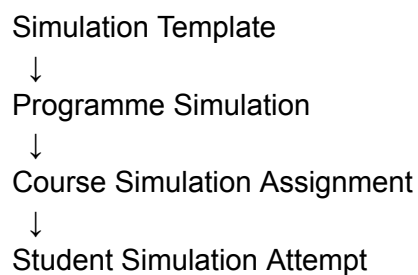


This hierarchy controls:

- access permissions;
 - dashboards;
 - reporting;
 - rubric aggregation;
 - competency analytics;
 - credit allocation;
 - LMS integration;
 - technical support routing;
 - grade return;
 - retry permissions;
 - longitudinal student tracking.
-

4. Simulation Hierarchy

The system should distinguish between reusable simulation templates, programme-level simulation designs, course-level simulation assignments, and individual student attempts.



Example

Template:

Initial ACT Intake — Anxiety and Avoidance

Used in:

- LI-CBT ACT Cohort 2026 — Course A
- LI-CBT ACT Cohort 2026 — Course B
- Advanced ACT Skills — Course C

Each course instance can define its own:

- dates;
 - teachers;
 - students;
 - challenge level;
 - permitted languages;
 - attempt limits;
 - grading policy;
 - LMS assignment link;
 - credit allocation;
 - support routing;
 - rubric version;
 - retry policy;
 - technical-failure policy.
-

5. Access Control Model

The system should use both:

1. **Role-based access control** — what type of user the person is.
 2. **Scope-based access control** — where in the hierarchy the user has access.
-

5.1 Core Roles

Role	Main Access
Student	Own simulations, grades, feedback, debrief chats, support requests
Teacher	Courses they teach, class dashboards, student attempts, course support cases
Programme Manager	All courses, simulations, dashboards, competency data, and support trends within their programme

College Manager	All programmes within their college, aggregate analytics, credit usage, support trends
Simulation Author	Can create/edit simulations depending on assigned scope
Support Staff	Can view technical tickets and diagnostic metadata according to assigned scope
System Admin	Platform-wide configuration, integrations, support management, security, billing, system health

A teacher may teach multiple courses.

A programme manager may also be a course teacher.

A college manager may have aggregate visibility across several programmes.

Support staff should not automatically receive access to full clinical transcripts, hidden case facts, student reflections, or full rubric evaluations unless explicitly authorised.

5.2 Access-Control Examples

Teacher A

- └─ Can see Course 1
- └─ Can see Course 4
- └─ Cannot see Course 2 or Course 3

Programme Manager

- └─ Can see all courses in Programme A
- └─ Cannot see Programme B

College Manager

- └─ Can see all programmes, courses, simulations, aggregate support trends, credit usage, and high-level quality indicators in their college.

5.3 Permission Matrix

Action	Student	Teacher	Programme Manager	College Manager	Support Staff	System Admin
Run assigned simulation	Yes	Demo optional	Demo optional	Demo optional	No	Yes

View own grade	Yes	—	—	—	No	Yes
View own feedback	Yes	—	—	—	No	Yes
View class dashboard	No	Own courses	Programme scope	College scope	No	Yes
View transcript	Own transcript	Own courses	Programme scope	College scope	Usually no	Yes
View class-wide rubric results	No	Own courses	Programme scope	College scope	No	Yes
Create simulation	No	If permitted	Yes	Yes	No	Yes
Edit simulation template	No	If permitted	Programme scope	College scope	No	Yes
Assign simulation to course	No	Own courses	Programme scope	College scope	No	Yes
Override AI grade	No	Own courses	Programme scope	College scope	No	Yes
View technical ticket	Own tickets	Course tickets	Programme tickets	College tickets	Assigned tickets	Yes
Escalate support issue	Yes	Yes	Yes	Yes	Yes	Yes
Authorise retry	No	Own courses	Programme scope	College scope	No	Yes
Export reports	No or limited	Own courses	Programme scope	College scope	No	Yes

6. Integration Model

The simulator should function as an independent platform with multiple entry points.

Entry Points

- |— Canvas LMS — self-hosted
- |— Moodle

└─ Secure standalone link

The same simulation engine, dashboards, rubrics, evaluations, multilingual logic, support system, and credit model should work regardless of entry method.

6.1 Canvas LMS Integration

Canvas integration is a core requirement.

Preferred approach:

Canvas self-hosted LMS



LTI 1.3 launch



Simulator platform



Student completes simulation



Evaluation generated



Optional grade / feedback return to Canvas

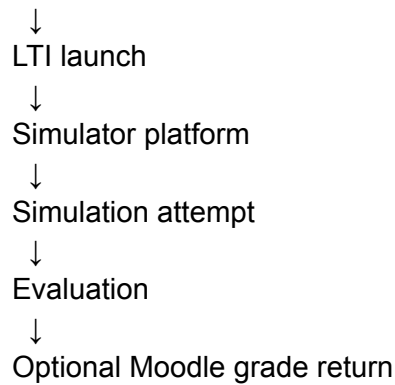
Canvas integration should support:

- LTI 1.3 launch;
 - Canvas user identification;
 - Canvas course context;
 - Canvas role detection;
 - Canvas assignment context;
 - Deep Linking for selecting simulations;
 - Assignment and Grade Services for grade return;
 - optional Names and Role Provisioning for roster sync;
 - diagnostic logging for failed launches and grade-sync problems;
 - technical support escalation linked to Canvas course and assignment context;
 - technical-failure status visible to teachers.
-

6.2 Moodle Integration

Moodle should be supported through LTI.

Moodle course activity



Moodle integration should support:

- LTI launch;
 - Moodle user identification;
 - Moodle course context;
 - Moodle role detection;
 - simulation assignment context;
 - grade return where configured;
 - diagnostic logging for launch and grade-sync failures;
 - technical support escalation linked to Moodle course/activity context;
 - technical-failure status visible to teachers.
-

6.3 Secure Link Mode

The platform should also support secure standalone link access.

This is useful for:

- pilots;
- workshops;
- external training;
- demos;
- colleges without LMS integration;
- private cohorts;
- future student-purchased practice.

Secure link access can use:

- invitation links;
- access codes;
- email login;
- magic link login;
- CSV roster upload;
- manual enrolment.

Even in secure link mode, the same hierarchy should remain:

College
↓
Programme
↓
Course / Group
↓
Simulation
↓
Student Attempt

7. LMS Launch Resilience

The Integration Layer must be designed for failure-tolerant LMS launch handling.

Some LTI errors may occur before the main React application loads.

Examples:

Possible LTI launch failures:

- |— invalid LTI signature
 - |— expired token
 - |— missing deployment/client key
 - |— incorrect Canvas/Moodle configuration
 - |— bad redirect URI
 - |— missing course/assignment context
 - |— role/context claim mismatch
 - |— course copy / assignment mapping problem
-

7.1 Fallback LTI Error Page

Because some LTI errors occur before the normal frontend application can boot, the backend must provide a lightweight fallback error page.

LTI launch attempt
↓
Backend validates launch
↓
If launch succeeds:
 load main simulator app
↓
If launch fails:

load fallback error page
allow support ticket creation

The fallback page should:

Fallback error page

- display a clear user-friendly error
- avoid exposing sensitive technical details to the user
- collect minimal diagnostic metadata
- allow user to add a short description
- allow “Contact support” immediately
- create a support ticket even when the main app fails
- route the issue to the configured support email

This prevents an LTI “catch-22” where the student cannot access the application and also cannot access support.

8. Frontend Diagnostic State

The frontend should maintain a global diagnostic state that can be attached to support tickets.

This is especially important in a MERN-style architecture where React state, backend events, and LMS launch context must be correlated.

8.1 Global Diagnostic State

The React frontend should maintain a global diagnostic snapshot.

GlobalDiagnosticState

- user role
- LMS source: Canvas / Moodle / Secure Link
- college / programme / course identifiers
- simulation assignment ID
- attempt ID
- current attempt status
- current route/screen
- microphone permission status
- dictation test status
- browser/device information
- network/API status
- last successful API response

- └─ last failed API response
- └─ last AI response status
- └─ last grade-sync status, if relevant
- └─ timestamp
- └─ client-side error summary

When the user clicks **Help**, the frontend should send a snapshot of this diagnostic state to the backend.

User clicks Help



Frontend captures diagnostic snapshot



Snapshot is sent to backend



Backend stores searchable metadata



Raw diagnostic log is stored separately



Support ticket is created or updated

The diagnostic snapshot should support both:

1. automated troubleshooting;
2. human escalation by email.

9. Competency Library

The system should include a layered competency library.

Core Competency Library



College Competency Library



Programme Competency Library



Course Competency Library



Simulation-Specific Criteria

9.1 Competency Levels

Level	Created / Managed By	Purpose
Core library	Platform admin / academic lead	Shared system-wide competency domains
College library	College manager	Institution-specific standards
Programme library	Programme manager	Programme-specific competencies
Course library	Teacher	Course-specific teaching emphasis
Simulation-specific criteria	Teacher / programme manager	Criteria relevant only to one simulation

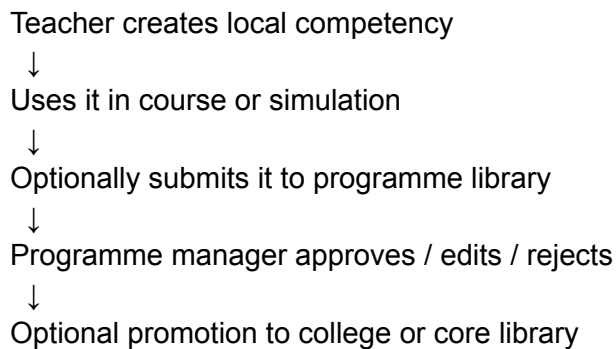
Teachers should be able to add competencies:

1. to a specific simulation;
2. to a specific course;
3. as a suggested addition to the programme or central library.

9.2 Competency Governance

Teacher-created competencies should not automatically become part of the central library.

Recommended workflow:



Competency statuses:

Draft
 Local course competency
 Simulation-specific criterion
 Submitted for review
 Approved for programme
 Approved for college
 Core / central
 Deprecated

9.3 Mapping Local Competencies

Local criteria should preferably be mapped to central competency domains.

Example:

Teacher-created criterion:

“The student notices that the client avoids contacting friends.”

Mapped to:

- functional_analysis
- avoidance_patterns
- ACT_case_conceptualisation

Important rule:

Mapped competencies

→ appear in longitudinal, course, programme, and college graphs

Unmapped local criteria

→ appear only in the specific course or simulation report

This allows teacher flexibility while preserving meaningful analytics.

10. Educator Case Authoring Engine

The platform should include a semi-automated case authoring system.

Teachers and programme managers should not need to write complex AI prompts manually.

Educator Case Authoring Engine

- Guided simulation builder
- Prompt-driven persona generation
- Diversity/context engine
- Clinical ground-truth boundaries
- Hidden information rules
- Resistance mapping
- Dynamic emotional-state settings
- Challenge-level settings
- Multilingual settings
- Rubric generator

- Simulation preview
- Authoring quality checker
- Version control

No batch case importer is included in this HLD.

10.1 Guided Simulation Builder

The builder should guide the educator through structured fields.

Field	Example
Simulation title	Initial ACT Intake — Anxiety and Avoidance
Clinical model	ACT / LI-CBT / CBT / mixed
Student level	Beginner / intermediate / advanced
Primary skill	Functional analysis
Secondary skill	Values clarification
Patient style	Guarded, polite, avoidant
Presenting problem	Anxiety before social interactions
Hidden issue	Avoidance is affecting work and relationships
Risk level	Low / moderate / high
Challenge level	1–5
Languages	Hebrew, English, Arabic later
Mode	Practice / formative / graded
Number of turns	50–75
Time limit	None / 15 / 30 / 50 minutes

10.2 Prompt-Driven Persona Generation

The system should generate realistic patient/client personas from educator inputs.

The persona generator should create:

- demographic profile;
 - contextual background;
 - communication style;
 - emotional presentation;
 - cultural/contextual considerations;
 - values conflict;
 - avoidance patterns;
 - help-seeking attitude;
 - resistance style;
 - likely disclosure pattern.
-

10.3 Inclusivity Principle

The diversity/context engine should support varied and culturally sensitive personas, while avoiding stereotype-based design.

Good design:

Cultural or demographic context

→ influences language, life context, barriers, values, and help-seeking behaviour only when educationally relevant.

Bad design:

Demographic identity

→ automatically determines pathology, personality, or risk.

The educator should be able to review and edit all generated persona details before publication.

11. Clinical Ground-Truth Boundaries

Every simulation should include a structured ground-truth file.

The AI patient may improvise natural language, emotion, hesitation, and style, but must not invent new case facts outside approved boundaries.

Case Ground Truth

- |— Known facts
- |— Presenting problem
- |— Symptoms / difficulties
- |— Timeline

- |— Functional impairment
- |— Maintaining processes
- |— Values conflicts
- |— What the patient knows
- |— What the patient does not know
- |— What may be disclosed
- |— What must not be invented
- |— Risk / safeguarding boundaries
- |— Escalation rules

Core rule:

The AI patient can vary expression, but not clinical reality.

This is essential for fair assessment.

12. Hidden Triggers and Resistance Mapping

The simulation should include programmable behavioural rules.

The patient should not reveal all information automatically. Some information should emerge only when the student demonstrates appropriate skill.

Example disclosure logic:

If student validates experience
→ patient becomes slightly more open

If student asks a specific behavioural question
→ patient gives more concrete detail

If student explores values respectfully
→ patient reveals value conflict

If student gives premature advice
→ patient becomes more guarded

If student pressures the patient
→ patient changes subject or becomes defensive

If student misses an emotional cue
→ patient becomes quieter or vague

This allows the simulation to assess clinical skill rather than only student persistence.

13. Dynamic Emotional-State Tracking

The simulator should maintain an internal patient state throughout the conversation.

- Patient State
- trust
 - openness
 - emotional activation
 - avoidance level
 - defensiveness
 - alliance quality
 - disclosure readiness
 - risk/safety relevance

The patient response should depend on this state.

Example:

- Student is empathic and specific
- trust increases
 - patient gives richer answer
- Student is rushed or judgmental
- defensiveness increases
 - patient becomes brief, vague, or evasive
-

14. Challenge Level

Teachers should be able to define how challenging the patient is.

The visible control can be simple:

Challenge level: 1–5

Underneath, the challenge level should affect multiple dimensions.

Dimension	Low Challenge	High Challenge
Openness	Shares easily	Guarded and vague

Emotional intensity	Calm	Distressed or reactive
Complexity	One clear issue	Multiple interacting issues
Insight	Understands patterns	Confused or externalising
Alliance	Cooperative	Tests the therapist
Risk ambiguity	None	Indirect hints
Disclosure	Reveals quickly	Reveals only after trust
Structure	Follows questions	Tangential or scattered

Example:

Challenge Level 4

- |— more vague answers
- |— slower disclosure
- |— stronger avoidance
- |— more emotional shifts
- |— more indirect risk signals
- |— greater need for precise therapeutic skill

15. Non-Verbal Cue Simulation

The AI patient should be able to include controlled non-verbal cues in text.

Examples:

[Looks away]
 [Long pause]
 [Sighs]
 [Smiles nervously]
 [Answers quickly, then changes the subject]

These cues should:

- appear only when relevant;
- reflect the patient's emotional state;
- be visible in the transcript;
- be available for evaluation;
- not become excessive or distracting.

Example evaluation use:

The patient looked away and became vague after the student moved too quickly into advice. The student did not notice or respond to this cue.

16. Supplementary Clinical Information Module

Instead of focusing on medical vitals or lab tests, the platform should support optional supplementary information relevant to psychotherapy training.

Examples:

- referral letter;
- previous treatment summary;
- symptom questionnaire;
- risk screening summary;
- values questionnaire;
- avoidance inventory;
- school/work functioning note;
- attendance or engagement history.

Teachers can decide whether students may request these during the simulation.

17. Student Simulation Interface

The student interface should support:

Student Simulation Interface

- Text chat
 - Voice dictation
 - Optional AI patient audio
 - Timer
 - Pause & reflect mode
 - Clinical notes side panel
 - Formulation side panel
 - Simulation instructions
 - Technical support button
 - Finish simulation button
 - Post-simulation feedback
-

17.1 Text and Voice Input

The core interface should be text-based, with optional dictation.

Recommended flow:

Student speaks



Audio is transcribed



Text appears in input box



Student can edit



Student sends response

This gives realism while preserving student control and accurate assessment.

17.2 Optional AI Patient Audio

In a later phase, the patient's response may be played aloud using text-to-speech.

Levels:

Level 1:

Student dictates; patient replies in text.

Level 2:

Student dictates; patient replies in text + audio.

Level 3:

Full real-time voice conversation.

MVP should support dictation first. Patient audio can be phase 2.

17.3 Time-Bound Sessions

Teachers should be able to configure time limits.

Options:

- no limit;
- 15 minutes;
- 30 minutes;
- 50 minutes;
- custom.

Assessment mode may use a hard stop. Practice mode may allow pause.

17.4 Pause & Reflect Mode

The system should include a configurable pause mode.

During pause, students can:

- review the conversation;
- write notes;
- draft the next question;
- consult an AI tutor if allowed;
- reflect on formulation;
- plan next steps.

Mode	Pause Allowed?	AI Tutor Allowed?
Practice	Yes	Yes
Formative	Optional	Optional
Graded	Teacher setting	Usually no
Exam	Usually no	No

17.5 Student Notes / Formulation Side Panel

Students should have a clinical workspace during the simulation.

For ACT / LI-CBT, this could include:

Clinical Workspace

- presenting problem
- situation / trigger
- thoughts
- emotions
- body sensations
- behaviour / avoidance
- short-term consequences
- long-term consequences
- values
- committed action possibilities
- risk / ethics notes
- hypotheses

└─ planned next question

Teachers can decide whether this workspace is visible in the final evaluation.

18. Simulation Runtime Model

The simulation should not work as a simple chatbot.

Instead, it should use a structured runtime process.

Student message



Interaction analyser



Update patient state



Check ground-truth and disclosure rules



Check resistance / trigger rules



Generate patient response



Store transcript + state changes

The interaction analyser should evaluate:

- empathy;
- question type;
- specificity;
- validation;
- ACT consistency;
- premature advice;
- pressure;
- missed cues;
- risk/ethics relevance;
- therapeutic stance.

The result affects the patient's next response.

19. Rubric Builder

Each simulation should have a semi-automatically generated rubric.

The rubric should be built from:

- simulation goals;
- clinical model;
- student level;
- challenge level;
- central competency library;
- programme competencies;
- teacher-added criteria;
- simulation-specific criteria.

Rubric flow:

Teacher defines simulation



System suggests rubric



Teacher edits criteria, weights, and wording



System generates scoring anchors



Teacher approves



Rubric version is locked for assigned simulation

Example rubric:

Criterion	Source	Weight
Builds therapeutic alliance	Core library	15
Clarifies the presenting problem	Core library	15
Identifies avoidance patterns	Programme ACT library	15
Uses ACT-consistent stance	Programme ACT library	20
Explores values	Core / ACT library	10
Notices risk / ethical boundaries	Core library	10
Structures the session effectively	Core library	10
Responds to guardedness	Simulation-specific	5

20. Evaluation and Debriefing Engine

At the end of the simulation, the system should generate structured evaluation.

Evaluation Engine

- |— Rubric scoring
- |— Criterion-level feedback
- |— Transcript evidence
- |— Highlighted strengths
- |— Highlighted missed opportunities
- |— Student-facing summary
- |— Teacher-facing report
- |— Remediation suggestions
- |— Student debrief chat
- |— Optional Canvas/Moodle grade return

The system should first generate structured data, then language.

Example structure:

```
{  
  "criterion_id": "functional_analysis",  
  "score": 6,  
  "evidence": "The student explored triggers but did not clarify consequences.",  
  "missed_opportunity": "The client hinted at avoidance after turn 14.",  
  "suggested_next_step": "Ask more concrete questions about what the client does when anxiety appears."  
}
```

20.1 Transcript Highlighting

The report should highlight specific moments in the conversation.

Examples:

Strong moment:

Turn 8 — Student asked an open and non-leading question.

Missed opportunity:

Turn 17 — Patient hinted at avoidance, but the student moved directly to advice.

Risk/ethics flag:

Turn 24 — Patient made an indirect safety-relevant statement that was not explored.

This makes feedback transparent and teachable.

20.2 Student Feedback

At the end of a simulation, students should see:

Student Feedback

- Overall grade
- Criteria-by-criteria score
- Strengths
- Growth areas
- Transcript examples
- Suggested alternative phrasing
- Recommended next practice focus
- Option to chat with evaluator

Example:

Criterion	Score	Feedback
Alliance	8/10	You responded warmly and validated the client's experience.
Problem mapping	6/10	You explored the problem, but did not sufficiently clarify triggers, behaviours, and consequences.
ACT stance	7/10	You used values-oriented language, but moved too quickly into advice.
Risk awareness	4/10	The client hinted at possible risk, but you did not explore safety sufficiently.

20.3 Post-Simulation Debrief Chat

After receiving feedback, the student should be able to chat with a debriefing tutor.

This is not the patient. It is an educational supervisor.

Students can ask:

Why did I lose points on functional analysis?
What would have been a better question?
Where did the client hint at avoidance?
How could I have explored values better?
Show me an example of a stronger response.
Did I miss any risk indicators?

Guardrails:

- uses only transcript, rubric, and evaluation;
 - does not invent new clinical facts;
 - does not change the official grade;
 - explains feedback pedagogically;
 - can cite relevant transcript moments;
 - can be visible to the teacher in graded mode.
-

20.4 Remediation Suggestions

The system should suggest targeted learning based on weak competencies.

Examples:

If weak in functional analysis:

→ suggest micro-learning on triggers, behaviours, and consequences.

If weak in values exploration:

→ suggest values clarification examples.

If weak in risk awareness:

→ suggest review of appropriate risk/safety inquiry within training scope.

If weak in therapeutic stance:

→ suggest examples of validation and open questions.

Remediation can include:

- short reading;
 - flashcards;
 - example questions;
 - practice simulation;
 - teacher-assigned review task.
-

21. Multilingual Design

The system should be multilingual from the beginning.

21.1 Student Language

Each student should be able to choose the simulation language, where enabled.

Example:

Student A performs in Hebrew.

Student B performs in English.

Student C performs in Arabic in the future.

21.2 Lecturer Language

Lecturers should be able to view:

- original transcript;
 - translated transcript;
 - evaluation in their chosen language;
 - rubric labels in their chosen language;
 - dashboard interface in their chosen language.
-

21.3 Language-Independent Analytics

Scores and competency IDs should be language-independent.

Example:

Criterion ID:

functional_analysis

Hebrew:

ניתוח תפקודי

English:

Functional analysis

Arabic:

التحليل الوظيفي

This allows dashboards to aggregate data across languages.

22. Dashboards and Analytics

Dashboards should be graph-first wherever possible.

Principle:

Graph first



Insight summary



Table / drill-down



Transcript evidence

22.1 College Dashboard

Audience: college manager.

Shows:

- programme comparison;
- total students;
- total simulations completed;
- average score by programme;
- competency strengths and weaknesses;
- credit usage;
- risk/ethics flag trends;
- technical support trends;
- LMS launch/grade-sync issue trends;
- drill-down into programmes.

Recommended graphs:

- programme comparison bar chart;
 - simulation completion trend;
 - competency heatmap across programmes;
 - credit usage graph;
 - flag/risk trend over time;
 - support requests by issue type;
 - support requests by programme;
 - unresolved support tickets over time.
-

22.2 Programme Dashboard

Audience: programme manager.

Shows:

- course comparison;
- cohort progress;
- simulation completion rates;
- average rubric scores;
- repeated weak competencies;
- teacher/course-level overview;
- longitudinal programme trends;
- technical issue trends across programme courses.

Recommended graphs:

- course comparison graph;
 - competency heatmap across courses;
 - progress over time by cohort;
 - simulation difficulty vs performance;
 - distribution of student scores;
 - support requests by course;
 - repeated technical issues.
-

22.3 Course Dashboard

Audience: teacher.

Shows:

- class list;
- assigned simulations;
- completion status;
- class-wide rubric results;
- student progress;
- transcript review;
- AI evaluation review;
- teacher comments;
- Canvas/Moodle grade return status;
- support issues affecting students in the course;
- technically affected attempts;
- retry requests pending.

Recommended graphs:

- class-wide rubric heatmap;
- score distribution graph;
- completion status graph;

- criterion-by-criterion average;
- individual student progress line;
- flags requiring teacher review;
- grade-sync status chart;
- student support request count.

Example class-wide insight:

The class is strong in alliance and validation,
but weaker in functional analysis and risk awareness.

22.4 Student Dashboard

Audience: student and authorised teachers/managers.

Shows:

- completed simulations;
- grades by simulation;
- criteria-by-criteria feedback;
- progress over time;
- feedback reports;
- debrief chat history;
- transcript access if enabled;
- support ticket history;
- credit usage if personal practice is enabled.

Recommended graphs:

- overall score over time;
- competency growth trend;
- radar chart of current competency profile;
- comparison with class average if enabled;
- before/after comparison across simulations.

23. Longitudinal Student Analytics

Teachers should be able to see development over time per student.

Example:

Simulation	Alliance	Problem Mapping	ACT Stance	Risk Awareness	Total
-------------------	-----------------	------------------------	-------------------	---------------------------	--------------

Simulation 1	70	58	62	50	61
Simulation 2	75	63	68	55	70
Simulation 3	82	71	76	64	77
Simulation 4	84	74	79	72	80

The system should automatically generate insights such as:

Dana is improving steadily in alliance and ACT stance.

Functional analysis is still below class average.

Risk assessment improved after Simulation 3.

Recommended next simulation: moderate-risk intake scenario.

24. Automated Support Tools

The platform should include automated support tools for students, teachers, programme managers, college managers, simulation authors, and technical support staff.

Automated Support Tools

- Student learning support
- Teacher support
- Programme manager support
- College manager support
- Authoring support
- Technical support assistant
- Automated troubleshooting
- Support ticket creation
- Human escalation by email
- Support analytics

The key principle:

The platform should not only assess students.

It should actively support students, teachers, programme managers, college managers, simulation authors, and technical users through automated insights, alerts, recommendations, troubleshooting, and guided escalation.

24.1 Student Learning Support

Student support tools should include:

- simulation instructions;
- technical readiness check;
- microphone test;
- language selector;
- accessibility settings;
- pause & reflect support;
- post-simulation debrief chat;
- rubric explanation assistant;
- remediation recommendations;
- next-practice recommendation;
- progress summary.

Example:

You scored lower on functional analysis.

Recommended support:

1. Review the short guide on triggers, behaviours, and consequences.
 2. Practise asking concrete behavioural questions.
 3. Try the easier “avoidance mapping” simulation before retrying.
-

24.2 Teacher Support

Teacher support tools should include:

- class weakness detection;
- rubric interpretation assistant;
- suggested lesson focus;
- flagged attempts queue;
- auto-generated class summary;
- student progress summaries;
- suggested remediation groups;
- simulation difficulty analysis;
- grade-review assistant;
- exportable supervision notes;
- visibility of repeated technical problems affecting the class.

Example:

Class insight:

Students are strong in validation and alliance, but weak in functional analysis.

Suggested next teaching focus:

Run a 20-minute classroom exercise on identifying triggers, avoidance behaviours, and short-term consequences.

24.3 Programme Manager Support

Programme manager support tools should include:

- programme-wide competency trends;
 - course comparison assistant;
 - simulation quality alerts;
 - repeated weakness detection;
 - teacher support recommendations;
 - curriculum alignment report;
 - competency coverage map;
 - cohort progression summary;
 - technical problem trends by course;
 - support workload overview.
-

24.4 College Manager Support

College manager support tools should include:

- college-wide usage summary;
 - credit consumption forecast;
 - programme comparison;
 - risk/ethics flag overview;
 - simulation completion trends;
 - quality assurance dashboard;
 - technical support trend dashboard;
 - automated management report.
-

24.5 Authoring Support

Simulation authoring support tools should include:

- case-quality checker;
- ground-truth consistency checker;
- rubric alignment checker;
- bias/stereotype warning;
- difficulty calibration assistant;
- language-readiness checker;

- missing-trigger detector;
- assessment-validity checker;
- simulation preview assistant.

Example:

Warning:

The simulation goal is “values clarification,” but only 5% of the rubric weight is assigned to values-related competencies.

Another example:

This simulation is marked beginner level, but includes:

- high emotional intensity
- guarded disclosure
- indirect risk hints

Suggested level: intermediate or advanced.

25. Technical Support Module

The platform should include a built-in technical support module to help users resolve common technical problems without immediately requiring human intervention.

The technical support system should combine:

Automated technical support



Context-aware troubleshooting



Support ticket creation



Escalation to human support by email

25.1 Technical Support Strategy

For MVP, technical support should rely primarily on deterministic decision trees rather than generative AI.

MVP technical support approach:

- └─ rule-based troubleshooting flows
- └─ predefined issue categories
- └─ structured diagnostic checks

- known recovery actions
- email escalation
- limited or no LLM-generated technical advice

This reduces:

- API cost;
- unpredictability;
- risk of inappropriate guidance;
- support inconsistency.

Generative AI may be added later for support summarisation, but only after strict boundaries are in place.

25.2 Support Assistant and Patient Simulator Separation

If an LLM-driven support assistant is added later, it must be completely separate from the AI patient simulator.

AI Patient Simulator

- patient role prompt
- case ground truth
- transcript context
- dynamic emotional state
- clinical simulation behaviour

Technical Support Assistant

- technical troubleshooting prompt
- diagnostic metadata
- support knowledge base
- issue routing rules
- no access to clinical roleplay context by default

The technical support assistant must not discuss:

Support assistant must not discuss:

- clinical case facts
- hidden patient information
- therapeutic strategy
- assessment interpretation
- how to improve clinical intervention
- rubric disputes
- student performance feedback

Example:

Student:

“The patient is not responding to my CBT interventions.”

Correct support behaviour:

“This may be an academic or simulation-content issue rather than a technical issue.
I can route this to your teacher/programme team.”

Incorrect support behaviour:

“Try using this CBT intervention instead...”

Clinical, academic, or rubric-related issues should be routed to the teacher or programme manager, not handled by technical support.

25.3 Technical Support Purpose

The technical support module should reduce support workload by automatically helping users resolve common problems such as:

- login or access problems;
 - Canvas launch problems;
 - Moodle launch problems;
 - secure link access problems;
 - microphone or dictation problems;
 - simulation not loading;
 - AI response delay or failure;
 - grade not appearing in the LMS;
 - missing simulation assignment;
 - language/display problems;
 - permission/access-control problems;
 - credit or usage problems;
 - browser compatibility problems.
-

25.4 Context-Aware Technical Support Assistant

The system should include a technical support assistant available from all major user screens.

Example locations:

Student simulation screen
Teacher dashboard
Programme dashboard
College dashboard
Simulation authoring screen
Login / access page
Post-simulation feedback screen
Technical support dashboard
Fallback LTI error page

The assistant should be context-aware.

For example, if a student opens support during a Canvas-launched simulation, the support assistant should already know:

- user role: student
- LMS source: Canvas
- course ID
- simulation assignment ID
- attempt ID
- browser/device information
- whether microphone permission was granted
- latest system error, if any
- grade-sync status, if relevant

This allows the system to offer relevant troubleshooting rather than generic help.

25.5 Automated Troubleshooting Flows

The support assistant should include structured troubleshooting flows.

Microphone / Dictation Problem

Problem: Dictation is not working

↓

Check browser microphone permission

↓

Check whether audio is being detected

↓

Offer test recording

↓

Suggest browser/device fix

↓

Allow student to continue with typed input

↓
Escalate if unresolved

LMS Launch Problem

Problem: Simulation does not open from Canvas/Moodle

↓
Check launch token/session
↓
Check user role
↓
Check course/simulation assignment mapping
↓
Check whether assignment is active
↓
Show user-friendly explanation
↓
Escalate to human support if unresolved

Grade Sync Problem

Problem: Grade not visible in Canvas/Moodle

↓
Check whether evaluation was completed
↓
Check whether teacher approval is required
↓
Check whether grade return was enabled
↓
Check last LMS grade-sync attempt
↓
Retry sync if allowed
↓
Escalate to support with technical log

Simulation Loading Problem

Problem: Simulation is stuck or not loading

↓
Check active attempt status
↓
Check AI service response status
↓
Check browser/network status
↓
Offer reload/resume attempt

↓
Preserve existing transcript
↓
Escalate if unresolved

25.6 Human Support Escalation by Email

If automated support cannot resolve the problem, the user should be able to escalate to human support by email.

The user should not need to manually explain all technical details.

The email should include:

Support escalation email

- |— user name
 - |— user email
 - |— user role
 - |— college / programme / course
 - |— LMS source: Canvas / Moodle / Secure Link
 - |— simulation assignment
 - |— attempt ID, if relevant
 - |— problem category
 - |— user-written description
 - |— automated troubleshooting steps already tried
 - |— browser/device information
 - |— timestamp
 - |— relevant error message
 - |— support log reference
-

25.7 Escape Hatch

The user should never be forced to complete the full automated troubleshooting flow before contacting a human.

Every support screen should include a visible:

Skip to Email Support

button.

Support flow:

User opens Help



System shows:

- likely issue
- quick recovery options
- Skip to Email Support



User may try automated troubleshooting



If unresolved, user escalates



System prepares structured support email



User may add free-text description



Ticket and email are sent

25.8 Frictionless Recovery

Wherever possible, technical problems should not block the learning task.

Examples:

Dictation problem:

→ offer typed input immediately.

Patient audio problem:

→ continue with text response.

Grade sync problem:

→ save evaluation internally and retry grade sync later.

Temporary AI response failure:

→ preserve transcript and allow safe retry.

Simulation loading issue:

→ offer resume attempt without losing prior transcript.

The system should prioritise keeping the student in the learning flow.

25.9 Support Confirmation and Expectation Management

After escalation, the confirmation screen must clearly explain what happens next.

The confirmation should include:

Support confirmation

- |— ticket ID
- |— who was notified
- |— expected response time
- |— whether the student may continue
- |— whether the teacher was notified
- |— whether the attempt is marked as technically affected
- |— what the user should do next

Example:

Your support request has been sent to Platform Technical Support.

Ticket ID: SUP-2026-00142

Expected response time: within 1 business day.

You can continue the simulation using typed input.

Your current transcript has been saved.

For academic-impact issues:

Your instructor has been notified.

This issue has been marked as affecting your simulation attempt.

Your teacher can review the attempt and allow a retry if needed.

25.10 Support Email Routing

Support emails should be routed according to deterministic configuration.

Issue Type	Routed To
Student access issue	Course teacher or technical support
LTI launch issue	Technical support

Grade sync issue	Teacher + technical support
Payment/credit issue	Billing/support
Permission issue	Programme manager or technical support
Platform bug	Technical support
Clinical/content issue	Teacher or programme manager
Rubric/evaluation dispute	Teacher
Simulation authoring issue	Programme manager or simulation author support
Academic impact due to technical failure	Teacher + technical support

Routing safeguards:

Routing safeguards:

- issue category required before escalation
- support email derived from scope and issue type
- fallback email only if configured
- audit log for all routing decisions
- tests for each routing rule
- no transcript attachment by default

The system should allow each college/programme/course to define support emails where appropriate.

26. Academic Safety for Technical Failures

Technical anxiety and platform instability can affect clinical performance. Therefore, the system must distinguish between normal incomplete attempts and attempts affected by confirmed technical errors.

26.1 Technically Affected Attempt Status

Add formal attempt statuses:

Attempt status:

- Not started
- In progress

- |— Completed
 - |— Abandoned by student
 - |— Submitted
 - |— Evaluated
 - |— Technically affected
 - |— Technical failure confirmed
 - |— Retry authorised
-

26.2 Academic Safety Flow

Technical issue detected during simulation



Support ticket created



Attempt is marked as technically affected



Teacher is notified where appropriate



If confirmed:

- attempt is marked technical failure confirmed

- student may be allowed retry

- LMS grade is not negatively affected

26.3 LMS Communication

Where supported, the platform should communicate technical failure status back to the LMS or make it visible to the teacher.

Possible Canvas/Moodle behaviours:

Possible LMS-facing behaviours:

- |— do not return failing grade for confirmed technical failure
- |— return “incomplete due to technical issue” comment, if appropriate
- |— notify teacher via dashboard
- |— allow teacher to authorise retry
- |— preserve audit trail

This protects students from being penalised for platform failure.

27. Grade Sync Design

Manual grade-sync retry is useful, but it must be idempotent.

27.1 Idempotent Grade Sync

The backend must ensure that repeated clicks do not create duplicate grade returns or conflicting LMS states.

Grade sync retry request



Backend checks existing sync state



If same grade already sent:
return current status



If sync in progress:
do not enqueue duplicate job



If previous sync failed:
enqueue one retry job



Record retry attempt

27.2 Grade Sync Fields

GradeSyncRecord

- |— id
- |— attempt_id
- |— lms_source
- |— lms_course_id
- |— lms_assignment_id
- |— user_id
- |— grade_value
- |— feedback_summary
- |— sync_status
- |— idempotency_key
- |— last_attempted_at
- |— retry_count
- |— last_error_code
- |— last_error_message

Suggested statuses:

Grade sync status:

- └─ Not required
 - └─ Pending
 - └─ In progress
 - └─ Synced
 - └─ Failed
 - └─ Retry queued
 - └─ Retry failed
 - └─ Blocked by technical failure
-

28. Diagnostic Log Strategy

Diagnostic logs can become large and should not be stored directly inside the main support ticket document.

28.1 Log Storage Principle

SupportTicket

→ stores searchable metadata and diagnostic_log_reference

Raw diagnostic logs

→ stored separately in NoSQL collection, object storage, or log pipeline

28.2 Recommended Storage Pattern

SupportTicket

- └─ issue metadata
- └─ routing metadata
- └─ status
- └─ priority
- └─ diagnostic_log_reference
- └─ no large raw log payload

DiagnosticLog

- └─ raw technical payload
- └─ React stack traces
- └─ API failure data
- └─ redacted LTI launch metadata
- └─ browser/device details
- └─ storage timestamp
- └─ retention policy

Raw logs may be stored in:

Options:

- |— separate NoSQL collection
 - |— object storage bucket
 - |— centralised logging service
 - |— secure application log pipeline
-

28.3 Redaction Requirement

Raw diagnostic logs must be redacted before storage where possible.

Avoid storing:

Avoid storing:

- |— full LTI tokens
 - |— raw private keys
 - |— access tokens
 - |— refresh tokens
 - |— full clinical transcript
 - |— student reflective notes
 - |— unnecessary personal data
-

29. Support Privacy Boundaries

Technical support should receive only what is required to solve the technical problem.

By default, support emails and support tickets should not include:

Do not include by default:

- |— full clinical simulation transcript
- |— hidden case facts
- |— student clinical formulation notes
- |— student reflective notes
- |— full rubric evaluation
- |— debrief chat
- |— sensitive student performance data

Allowed by default:

Allowed by default:

- |— user identity and contact information
- |— course/simulation identifiers

- attempt ID
- error category
- error code
- browser/device information
- microphone permission status
- LTI source/context metadata
- grade-sync status
- diagnostic log reference
- user-written description

If support staff need access to transcripts or evaluations, this should require explicit elevated permission and be recorded in an audit log.

Sensitive access controls:

- explicit permission required
 - reason for access recorded
 - access logged
 - time-limited access preferred
 - visible to authorised admin/auditor
-

30. Automated Alerts

The system should include automated alerts, but they must be carefully scoped.

Automated Alerts

- Student did not complete simulation
- Student repeatedly fails same competency
- Student triggers risk/ethics flag
- Class-wide weakness detected
- Simulation appears too easy/hard
- Unusual cost spike
- Repeated technical failures
- LMS launch failures
- Grade-sync failures
- Dictation/microphone failures
- Low remaining credit balance

Teacher alert example:

Five students in Course A missed indirect risk cues in the same simulation.

Suggested action:

Review risk-screening questions in the next class.

Technical alert example:

Course B has repeated Canvas launch failures.

Suggested action:

Check LTI assignment mapping and recent course-copy settings.

Cost alert example:

Simulation “Complex Intake 3” is using 2.4× the average token budget.

Suggested action:

Review prompt length, max turns, debrief settings, and transcript memory strategy.

31. Cost Governance Layer

The platform should include a cost governance layer to keep AI usage predictable.

Cost Governance Layer

- Per-simulation token budget
- Max student turns
- Max dictation minutes
- Max debrief-chat questions
- Max retry attempts
- Prompt caching
- Sliding-window memory
- Transcript summarisation
- Model routing
- Cost logging per attempt
- Cost dashboard
- College/programme/course credit limits

Recommended default limits:

Limit	Suggested Default
Max turns per simulation	75
Soft warning	60 turns
Max student audio minutes	15 minutes
Max final evaluation generations	1

Max failed-response retries	1
Max debrief questions	5–10
Max official attempts	Teacher setting
Extra practice attempts	Credit-based

31.1 Planning Cost Estimate

For planning purposes, assuming:

- 50–75 student turns per simulation;
- 2–3 sentence student messages;
- AI patient replies of moderate length;
- one final rubric-based evaluation;
- optional dictation;
- light post-simulation debrief chat;
- optimized memory management and model routing;

a reasonable planning estimate is:

Usage Model	Estimated Cost Per Simulation	Estimated Cost Per Student for 10 Simulations
Text only, optimized	~\$0.07–\$0.17	~\$0.70–\$1.70
Text + dictation	~\$0.09–\$0.21	~\$0.90–\$2.10
Text + dictation + light debrief chat	~\$0.10–\$0.24	~\$1.00–\$2.40
With safety buffer	~\$0.15–\$0.35	~\$1.50–\$3.50

Recommended internal budgeting assumption:

Budget approximately \$2–\$4 API cost per student for 10 simulations for a well-optimized MVP with dictation and debrief chat.

For institutional planning, round up to approximately \$5 per student per 10 simulations to cover longer sessions, retries, translation, failed requests, lecturer re-runs, and future feature expansion.

Precise vendor pricing should be rechecked before procurement and before commercial pricing is finalised.

31.2 Support Cost Logging

Technical support should be designed to minimise unnecessary AI cost.

MVP technical support:

Use deterministic troubleshooting first.

Use LLM only for optional summarisation or later advanced support.

If AI is later used for support, the usage should be logged separately from simulation costs.

SupportAIUsageLog

- id
- support_ticket_id
- model_used
- input_tokens
- output_tokens
- estimated_cost
- created_at

This keeps educational simulation cost separate from operational support cost.

32. Credit and Usage System

The platform should support future student-purchased credits, but separate formal assessment from personal practice.

Credit types:

Institutional credits

- └─ purchased/provided by college, programme, or course
- └─ used for assigned simulations
- └─ visible in management dashboards

Personal practice credits

- └─ purchased by student
- └─ used for extra practice
- └─ not automatically counted toward formal grade
- └─ optionally shareable with teacher

Credit hierarchy:

College credit pool

↓

Programme allocation



Course allocation



Student required attempts



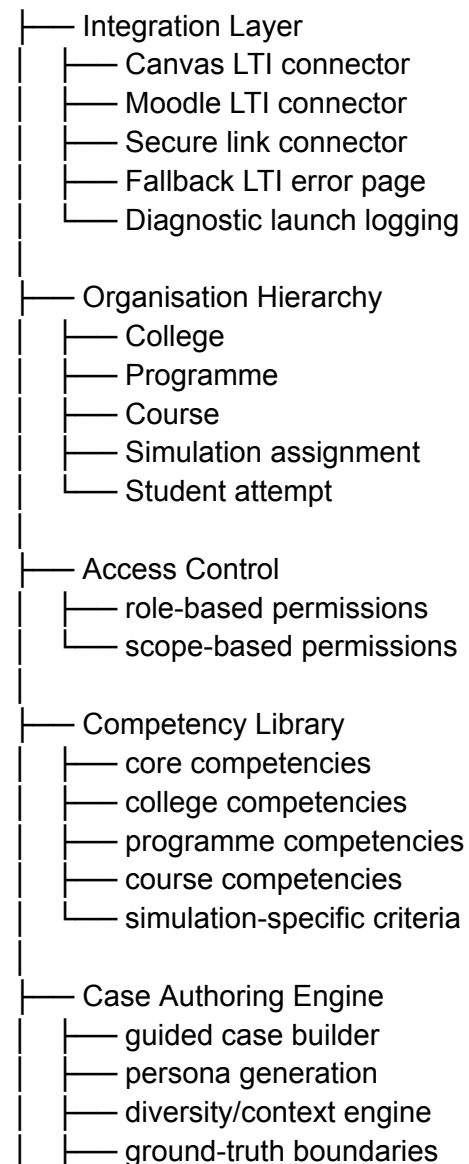
Optional student-purchased practice

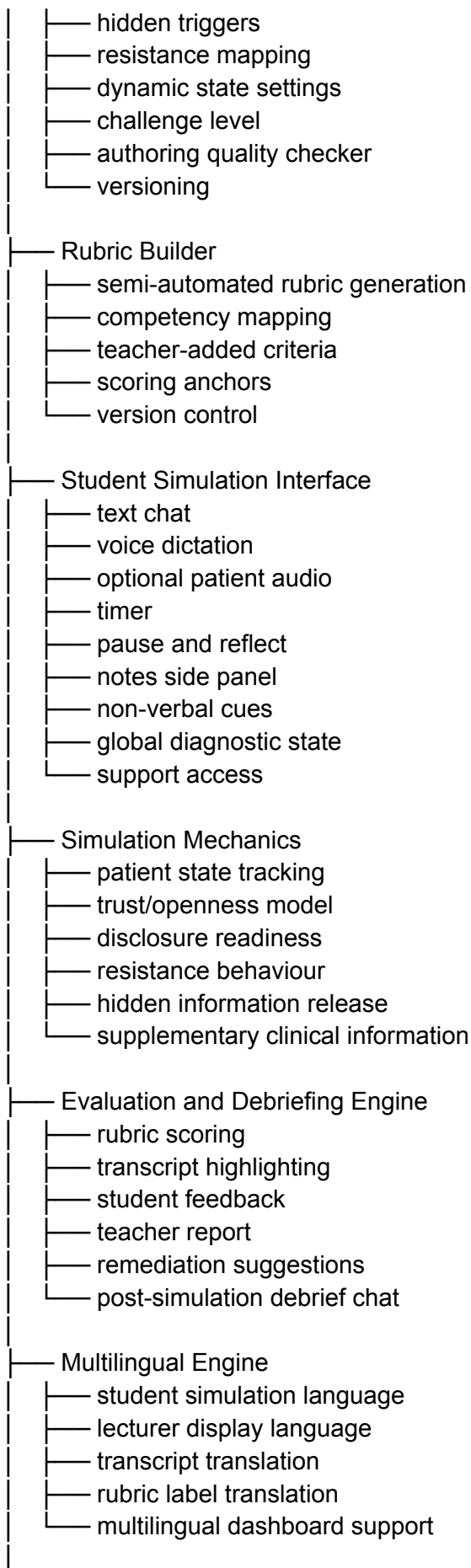
Important rule:

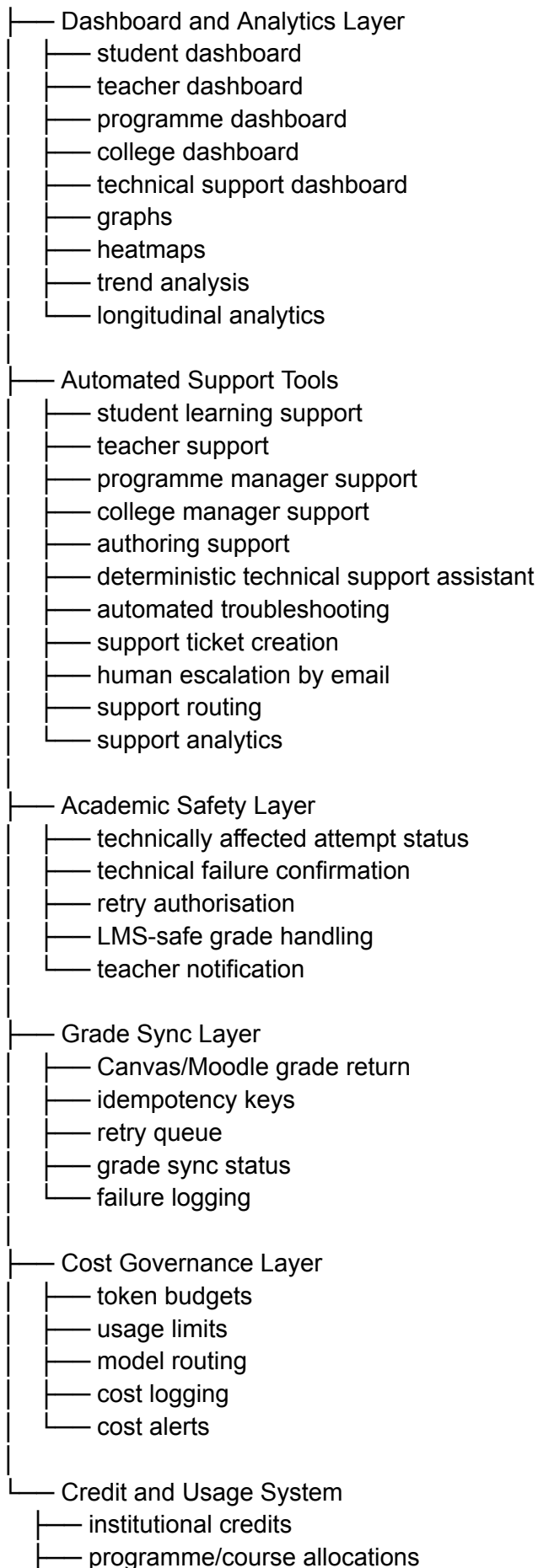
Student-purchased simulations should not automatically affect formal grades.

33. Main Platform Modules

AI Patient Simulator Platform







- └─ personal practice credits
 - └─ cost monitoring
-

34. Suggested High-Level Data Model

College

- id
- name
- settings
- default_language
- credit_pool
- support_email

Programme

- id
- college_id
- name
- competency_framework
- managers
- support_email

Course

- id
- programme_id
- canvas_course_id
- moodle_course_id
- name
- teachers
- students
- support_email

User

- id
- name
- email
- roles
- preferred_language

RoleAssignment

- user_id
- role
- scope_type
- scope_id

Competency

- id

- name
- description
- level
- parent_competency_id
- status
- language_labels
- created_by

SimulationTemplate

- id
- owner_scope
- title
- clinical_model
- template_json
- available_languages
- version
- status

SimulationAssignment

- id
- template_id
- course_id
- difficulty_settings
- rubric_id
- start_date
- due_date
- attempt_limit
- grading_policy
- support_policy
- retry_policy

Rubric

- id
- simulation_template_id
- competency_mapping
- criteria
- weights
- scoring_anchors
- version

Attempt

- id
- assignment_id
- student_id
- language
- status
- started_at
- completed_at

- score
- flags
- credit_source
- technical_impact_status

Message

- id
- attempt_id
- speaker
- original_language
- original_text
- translated_text
- non_verbal_cue
- timestamp

PatientStateLog

- id
- attempt_id
- turn_number
- trust
- openness
- defensiveness
- emotional_activation
- disclosure_readiness

Evaluation

- id
- attempt_id
- rubric_scores
- student_feedback
- teacher_report
- highlighted_transcript_moments
- remediation_suggestions
- generated_language
- approved_by_teacher

DebriefChat

- id
- attempt_id
- student_id
- messages
- visible_to_teacher

SupportTicket

- id
- user_id
- role
- scope_type

- scope_id
- college_id
- programme_id
- course_id
- lms_source
- issue_category
- user_description
- automated_steps_attempted
- status
- priority
- assigned_support_email
- created_at
- updated_at
- resolved_at
- related_attempt_id
- related_assignment_id
- diagnostic_log_reference
- academic_impact_status
- routed_to_role
- routed_to_email

DiagnosticLog

- id
- support_ticket_id
- user_id
- attempt_id
- assignment_id
- lms_source
- error_code
- error_message
- frontend_route
- browser_info
- device_info
- microphone_permission_status
- last_api_status
- last_ai_response_status
- last_grade_sync_status
- redacted_lti_context
- raw_payload_storage_uri
- created_at
- retention_until

GradeSyncRecord

- id
- attempt_id
- lms_source
- lms_course_id
- lms_assignment_id

- user_id
- grade_value
- feedback_summary
- sync_status
- idempotency_key
- last_attempted_at
- retry_count
- last_error_code
- last_error_message

CreditLedger

- id
- owner_type
- owner_id
- credit_type
- amount
- usage_reason
- timestamp

UsageLog

- id
- attempt_id
- model_used
- input_tokens
- output_tokens
- transcription_minutes
- estimated_cost
- timestamp

SupportAIUsageLog

- id
- support_ticket_id
- model_used
- input_tokens
- output_tokens
- estimated_cost
- created_at

34.1 Recommended Support Ticket Indexes

To support the technical support dashboard, the database should include indexes such as:

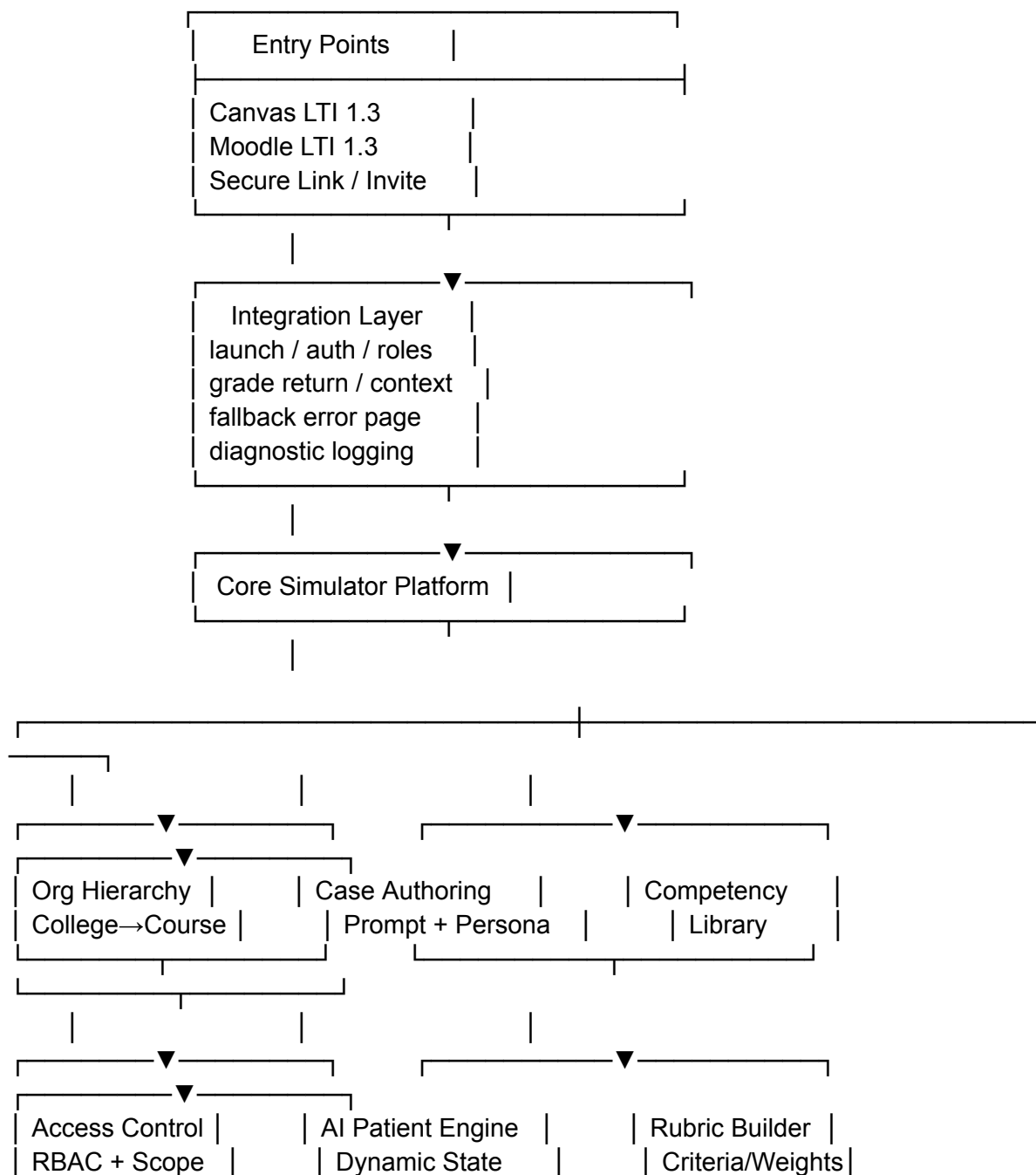
Recommended indexes:

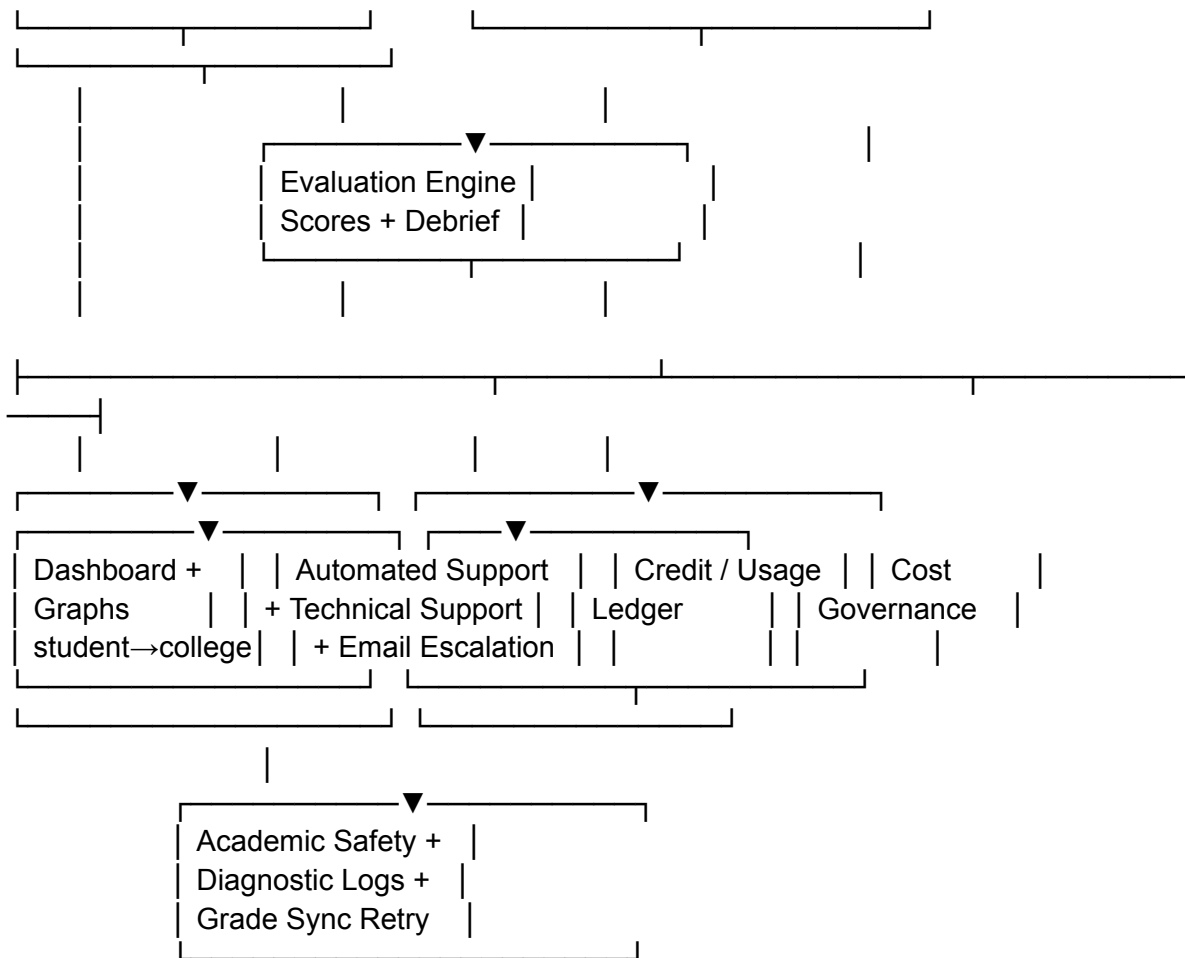
- |— [status, assigned_support_email]
- |— [created_at, issue_category]

- [college_id, programme_id, status]
- [course_id, issue_category]
- [related_attempt_id]
- [lms_source, issue_category, created_at]

These indexes support support queues, dashboards, issue trends, and escalation monitoring.

35. High-Level Architecture Diagram





36. QA and Testing Requirements

Testing must include controlled failure simulation.

36.1 QA Failure Simulation Tools

QA should be able to simulate:

QA simulation tools:

- blocked microphone permission
- denied microphone permission
- lost browser network connection
- AI API timeout
- AI API failure
- failed transcript save
- dropped Canvas/Moodle connection mid-session
- expired LTI token

- invalid LTI signature
 - missing LTI assignment mapping
 - failed grade sync
 - duplicate grade-sync retry click
 - support email routing failure
 - fallback error page rendering
-

36.2 QA Test Categories

QA test categories:

- LTI launch success/failure
 - fallback error-page support creation
 - diagnostic snapshot capture
 - support ticket creation
 - email routing
 - privacy redaction
 - grade-sync idempotency
 - technical-failure academic safety flow
 - frontend recovery without data loss
 - dictation fallback to typing
 - dashboard support metrics
 - role/scope access boundaries
-

36.3 Email Routing Regression Tests

The routing matrix must have regression tests.

Example tests:

Routing regression tests:

- student access issue goes to correct course support
 - grade sync issue notifies teacher + technical support
 - clinical/content issue does not go only to IT
 - platform bug does not expose transcript
 - payment issue does not notify teacher unnecessarily
 - permission issue routes by scope
 - fallback email is used only when no scoped email exists
-

37. MVP Scope

37.1 MVP Must Include

MVP core:

- |— College / programme / course / simulation / student hierarchy
 - |— Canvas self-hosted LTI launch
 - |— Fallback LTI error page
 - |— Secure standalone link mode
 - |— Moodle-ready integration architecture
 - |— Role and scope-based access control
 - |— Simulation template system
 - |— Guided case authoring
 - |— Ground-truth boundaries
 - |— Basic persona generation
 - |— Basic challenge-level settings
 - |— Text chat simulation
 - |— Student voice dictation
 - |— Basic non-verbal cues
 - |— Basic dynamic emotional-state tracking
 - |— Rubric-based evaluation
 - |— Student feedback by criterion
 - |— Teacher dashboard
 - |— Class-wide rubric graph
 - |— Transcript highlighting
 - |— Student post-simulation debrief chat
 - |— Basic automated student support
 - |— Basic teacher support insights
 - |— Deterministic technical support assistant
 - |— Skip to Email Support button
 - |— Email escalation to human support
 - |— Support ticket record
 - |— Separate diagnostic log reference
 - |— Global frontend diagnostic snapshot
 - |— Dictation fallback to typing
 - |— Basic technical-failure attempt status
 - |— Teacher visibility of technically affected attempts
 - |— Grade-sync idempotency protection
 - |— Basic support routing matrix
 - |— No full transcript in support ticket by default
 - |— QA tools for key simulated failures
 - |— Basic cost monitoring
-

37.2 MVP Optional, If Feasible

MVP optional:

- |— Pause & reflect mode
- |— Student notes/formulation side panel

- Basic remediation suggestions
 - Canvas grade return
 - Automated grade-sync retry
 - Simple programme dashboard
 - Support dashboard for system/admin users
 - Support dashboard graphs
 - Automated support trend summaries
 - Advanced diagnostic log pipeline
-

37.3 Phase 2

Phase 2:

- Full Moodle LTI grade return
 - Advanced graph dashboards
 - Programme and college dashboards
 - Advanced authoring studio
 - Advanced dynamic emotional state
 - Patient audio responses
 - Deeper multilingual translation workflow
 - Longitudinal analytics
 - Remediation content library
 - Advanced technical support dashboard
 - Support analytics by college/programme/course
 - Automated issue clustering
 - Advanced support routing rules
 - Support SLA management
 - Support AI for summarising tickets
 - Deeper LMS diagnostic tooling
 - Technical incident reports
 - Expanded QA failure simulation suite
-

37.4 Phase 3

Phase 3:

- Student-purchased credits
- Real-time voice conversation
- Advanced simulation marketplace/library
- Research export
- Benchmarking across cohorts
- Blackboard or additional LMS integrations
- Advanced support SLA management
- Human support team dashboard with workflow automation
- Advanced institutional reporting

38. Core Design Principles

38.1 Hierarchy First

The system must be designed from the start around:

College → Programme → Course → Simulation → Student Attempt

This affects permissions, dashboards, grades, analytics, credits, reporting, and support routing.

38.2 Competency Mapping Is Essential

Teacher flexibility is important, but analytics require shared competency mapping.

Flexible local criteria

+ central competency mapping

= meaningful longitudinal analytics

38.3 The AI Patient Must Have Boundaries

The patient can improvise wording and emotion, but not invent clinical facts outside the ground-truth file.

38.4 Simulation Is More Than Chat

The system should track patient state, trust, disclosure readiness, resistance, emotional activation, and non-verbal cues.

38.5 Feedback Must Be Evidence-Based

Every evaluation should connect to transcript evidence.

38.6 Student Debriefing Is Part of Learning

The post-simulation evaluator chat should help students understand their performance without changing the official grade.

38.7 Multilingual Support Must Be Structural

Translation should not be an afterthought. Competency IDs, rubric IDs, scores, and analytics must be language-independent.

38.8 Formal Assessment and Personal Practice Must Remain Separate

Future student-purchased credits should support extra learning, but not distort official course grades.

38.9 Failure Must Be Supported Before the App Boots

LMS launch failures may occur before the main frontend loads. The platform must therefore provide a fallback error page capable of creating support tickets independently of the main React app.

38.10 Technical Support Must Be Separate From Clinical Simulation

Technical support must not share the AI patient's prompt, transcript context, case facts, or clinical roleplay state.

38.11 Deterministic Support First

For MVP, support should rely on predictable troubleshooting flows and escalation rules, not open-ended generative support.

38.12 Always Provide a Human Escape Hatch

Users should always be able to skip directly to email support.

38.13 Technical Failure Must Not Become Academic Penalty

If a simulation is disrupted by confirmed technical failure, the attempt should be marked accordingly and the student should be protected from unfair grading consequences.

38.14 Support Privacy Is Essential

Technical support should not receive full transcripts, evaluations, clinical notes, hidden case material, or student reflections by default.

38.15 Grade Sync Must Be Idempotent

Manual or automated grade-sync retries must not create duplicate, conflicting, or repeated LMS grade submissions.

38.16 Logs Must Be Searchable but Not Bloated

Support tickets should contain searchable metadata. Large raw diagnostic logs should be stored separately with redaction and retention controls.

38.17 Cost Governance Is a Core Requirement

AI usage must be visible, bounded, and auditable at student, course, programme, and college levels.

39. Final HLD Summary

The AI Patient Simulator is a hierarchical, multilingual, LMS-integrated clinical simulation platform designed for psychotherapy and LI-CBT/ACT clinical skills training.

It supports:

- Canvas self-hosted integration;
- Moodle integration;
- standalone secure links;
- fallback LMS launch support;
- college/programme/course hierarchy;
- role and scope-based permissions;
- guided simulation authoring;
- ground-truth patient boundaries;
- dynamic emotional-state simulation;
- hidden triggers and resistance mapping;
- competency-linked rubrics;
- graph-based dashboards;
- student feedback by criterion;
- transcript-based evaluation;
- post-simulation debrief chat;
- automated learning support;
- teacher and programme support tools;
- integrated deterministic technical support;
- escalation to human support by email;
- privacy-protective diagnostic logging;
- technical-failure academic safety;
- idempotent LMS grade sync;
- cost governance;
- institutional and personal credit models;
- longitudinal student development tracking;
- QA tools for external-failure simulation.

Its central educational purpose is to help students practise, receive structured feedback, understand their development, and improve clinical competencies over time, while helping educators and institutions monitor learning quality, identify support needs, protect assessment fairness, and manage simulation-based training at scale.

Recommended Tools

Recommended tools

Recommended Stack

Stack:
Next.js + NestJS + TypeScript + PostgreSQL + Prisma + Redis + S3-compatible storage

AI development tools:
Cursor for daily coding
OpenAI Codex for autonomous feature branches
Claude Code for terminal/repo-level debugging and refactoring
GitHub Copilot Coding Agent for GitHub issue-to-PR work
Vercel v0 for UI generation
Playwright for end-to-end testing
GitHub Actions for CI/CD

Recommended Core development tools

Purpose	Recommended tool
Source control	GitHub
Issue/task management	GitHub Issues or Linear
Primary AI IDE	Cursor
Large autonomous coding tasks	OpenAI Codex
Terminal/repo debugging	Claude Code
GitHub PR agent	GitHub Copilot Coding Agent
UI generation	Vercel v0
Database	PostgreSQL
ORM	Prisma
Backend framework	NestJS
Frontend framework	Next.js

Testing	Playwright
CI/CD	GitHub Actions
Error monitoring	Sentry-style monitoring
Email sending	Resend / SendGrid / AWS SES
Object storage	S3-compatible storage
Queue	Redis + BullMQ

Playwright is especially suitable because it supports modern end-to-end web testing and can automate Chromium, Firefox, and WebKit. It also describes support for testing, scripting, and AI-agent workflows.

Recommended development stages

Stage 1 — Foundation

- monorepo setup
- Next.js app
- NestJS API
- PostgreSQL schema
- Prisma
- role/scope model
- basic dashboard shell
- secure-link login

Stage 2 — Simulation core

- simulation templates
- attempt lifecycle
- AI patient engine
- transcript storage
- ground-truth boundaries
- dynamic patient state

Stage 3 — Evaluation

- rubric builder
- structured scoring
- transcript highlights
- student feedback
- teacher report
- debrief chat

Stage 4 — Dashboards

- teacher dashboard
- class-wide rubric graph
- student longitudinal view
- support indicators

Stage 5 — Canvas LTI

- LTI launch
- role mapping
- assignment context
- fallback error page
- grade sync
- idempotent retries

Stage 6 — Technical support

- global diagnostic state
- support ticket
- deterministic flows
- email escalation
- support routing
- technical-failure academic safety

Stage 7 — Moodle and scale

- Moodle LTI
- programme dashboard
- college dashboard
- cost governance
- credit system